TIME COMPLEXITY & SPACE COMPLEXITY

Study Notes | FY MCA

1. Introduction

An algorithm is a finite sequence of steps used to solve a problem. Two algorithms can produce the same result but use different amounts of time and memory.

Algorithm complexity describes how the resources required by an algorithm grow as the input size increases.

- Time complexity: how the number of basic operations grows with input size.
- Space complexity: how the memory requirement grows with input size.

Complexity is usually analysed mathematically; it is not the same as measuring the program's actual running time in seconds, which also depends on hardware, programming language, compiler, and implementation.

2. What does n mean?

The variable n usually represents the size of the input.

- Array: number of elements
- String: number of characters
- File: amount of data or number of records
- Graph: often represented using V vertices and E edges

If an algorithm processes every array element once, its work generally grows with n.

3. Big-O notation

Big-O notation describes an upper bound on the growth rate of an algorithm, commonly used to discuss its asymptotic worst-case time or space complexity. In introductory analysis, it helps us compare how algorithms scale for large inputs.

Simplification rules

- Ignore constant factors: $O(5n) = O(n)$, and $O(100) = O(1)$.
- Keep the dominant term: $O(n^2 + n + 10) = O(n^2)$.
- Logarithm bases differ only by a constant factor: $O(\log_2 n) = O(\log_{10} n) = O(\log n)$.

Constant time does not mean exactly one operation. It means the amount of work does not grow with n.

4. Increasing order of common time complexities

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(n^k) < O(2^n) < O(n!)$$

This is a common growth-rate hierarchy for sufficiently large n. Here, k is a fixed positive constant. $O(n^k)$ represents polynomial complexity; $O(2^n)$ is exponential; $O(n!)$ is factorial.

5. Common complexities and examples

5.1 O(1) — Constant time

```
int x = arr[5];
```

Accessing a known array index takes constant time in the usual array model. The operation does not grow with the number of array elements.

Typical pattern: a fixed number of operations, independent of n.

5.2 O(log n) — Logarithmic time

```
int i = n;
while (i > 1) {
    i = i / 2;
}
```

The value is repeatedly halved: n, n/2, n/4, n/8, …, 1. The number of halvings is approximately $\log_2 n$, so the complexity is O(log n).

```
for (int i = 1; i < n; i = i * 2) {
    printf("%d", i);
}
```

Here the values double: 1, 2, 4, 8, 16, … . The number of iterations is logarithmic.

Useful clue: repeatedly multiplying or dividing a loop variable by a constant often indicates O(log n), provided the loop progresses toward its stopping condition.

5.3 O(√n) — Square-root time

```
for (int i = 1; i * i <= n; i++) {
    printf("%d", i);
}
```

The condition $i^2 \le n$ allows i to grow only up to approximately $\sqrt{n}$. Therefore, the loop executes $O(\sqrt{n})$ times.

5.4 O(n) — Linear time

```
for (int i = 0; i < n; i++) {
    printf("%d", i);
}
```

The body executes n times, so the time complexity is O(n). Doubling n approximately doubles the number of iterations.

5.5 Sequential loops: add

```
for (int i = 0; i < n; i++) { /* work */ }
for (int j = 0; j < n; j++) { /* work */ }
```

Total work is n + n = 2n. Dropping the constant factor gives O(n), not $O(n^2)$. Sequential blocks are added before simplification.

5.6 O(n log n) — Linearithmic time

```
for (int i = 0; i < n; i++) {
   for (int j = 1; j < n; j = j * 2) {
       printf("*");
    }
}
```

The outer loop runs n times. For each outer iteration, the inner loop runs O(log n) times. Total work is n × log n, giving O(n log n).

Merge sort and heap sort have O(n log n) time complexity in their standard implementations; quicksort has O(n log n) average time but O(n²) worst-case time.

5.7 O(n²) — Quadratic time

```
for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
        printf("*");
    }
}
```

The inner loop runs n times for each of the n outer iterations: n × n = n². Therefore, the complexity is O(n²).

5.8 O(n³) — Cubic time

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        for (int k = 0; k < n; k++) {
           printf("*");
      }
    }
}
```

There are three nested loops, each running n times. Total work is n × n × n = n³, so the complexity is O(n³). This rule applies when each loop really does run n times.

5.9 Nested loops do not always mean O(n²)

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 10; j++) {
        printf("*");
    }
}
```

The outer loop runs n times, while the inner loop runs a fixed 10 times. Total work is 10n, which simplifies to O(n). Always count executions rather than merely counting loops.

5.10 O(n + log n) simplifies to O(n)

```
for (int i = 0; i < n; i++) { /* work */ }
int x = n;
while (x > 1) { x = x / 2; }
```

The first loop takes O(n); the second takes O(log n). Total: O(n + log n). Since n grows faster than log n, the dominant term is n, so the simplified complexity is O(n).

5.11 Polynomial, exponential, and factorial

- Polynomial: $O(n)$, $O(n^2)$, $O(n^3)$, …, $O(n^k)$, where k is a fixed constant.
- Exponential: $O(2^n)$. A common recursive pattern makes two recursive calls on a problem of size n − 1; its total time is $O(2^n)$.
- Factorial: O(n!). Generating all permutations of n distinct elements requires examining n! arrangements, apart from the cost of producing each arrangement.

```
void fun(int n) {
    if (n == 0) return;
    fun(n - 1);
    fun(n - 1);
}
```

For this recursive example, the time complexity is $O(2^n)$, and the maximum recursion-stack space is O(n), assuming each call uses constant additional space.

6. Time complexity calculation method

1. Identify the input size (n, or separate sizes such as n and m).
2. Choose the basic operation to count, such as a print, comparison, or assignment.
3. Count how many times the operation executes.
4. For sequential blocks, add the work of each block.
5. For nested loops, determine the iterations of each loop and multiply when the iteration counts are independent.
6. Write the expression, such as $3n^2 + 5n + 10$.
7. Drop constant factors and keep the dominant term.
8. Write the final Big-O complexity.

7. Space complexity

Space complexity describes how memory usage grows with input size. Auxiliary space means the extra memory used by an algorithm, often excluding the input storage itself. Be clear about which convention a question uses.

O(1) auxiliary space

```
int total = 0;
for (int i = 0; i < n; i++) {
    total += i;
}
```

The algorithm uses a fixed number of variables. The loop takes O(n) time, but the auxiliary space is O(1).

O(n) space

```
int *arr = allocate_array_of_size(n);
```

An array containing n elements uses O(n) space.

$O(n^2)$ space

```
int **matrix = allocate_matrix(n, n);
```

An n × n matrix stores n^2 elements, so its space complexity is $O(n^2)$.

Recursion and the call stack

```
void fun(int n) {
    if (n == 0) return;
    fun(n - 1);
}
```

There are O(n) calls over the full execution and up to O(n) active stack frames. Therefore, time is O(n) and auxiliary stack space is O(n).

8. Quick reference table

Pattern	Time complexity	Reason
Fixed number of operations	O(1)	Does not grow with n
Repeated doubling or halving	O(log n)	Logarithmic number of steps
Loop up to √n	O(√n)	Approximately √n iterations
Single loop to n	O(n)	n iterations
Two separate loops to n	O(n)	n + n = 2n
Loop to n containing a log n loop	O(n log n)	n × log n
Two nested loops, each to n	$O(n^2)$	n × n
Three nested loops, each to n	$O(n^3)$	n × n × n
Fixed power n^k	$O(n^k)$	Polynomial
All permutations	O(n!)	n! arrangements

9. Practice questions

Instructions: For each code snippet, calculate (a) time complexity and (b) auxiliary space complexity. Assume each displayed body operation takes constant time and use standard array-operation assumptions. If only time is requested, simplify to Big-O.

Q1. Identify the time complexity.

```
int x = arr[3];
```

Q2. Identify the time complexity.

```
for (int i = 0; i < n; i++) {
    printf("Hi");
}
```

Q3. Identify the time complexity.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        work();
    }
}
```

Q4. Identify the time complexity.

```
for (int i = 1; i < n; i *= 2) {
  work();
}
```

Q5. Identify the time complexity.

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j *= 2) {
       work();
    }
}
```

Q6. Simplify the complexity.

```
for (int i = 0; i < n; i++) work();
for (int j = 0; j < n; j++) work();
```

Q7. Identify the time complexity.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 10; j++) work();
}
```

Q8. Simplify the expression.

```
O(n² + 4n + 20)
```

Q9. Simplify the expression.

```
O(n + log n)
```

Q10. Identify the time complexity.

```
for (int i = 1; i * i <= n; i++) work();
```

Q11. Identify the time complexity.

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) work();
}
for (int k = 0; k < n; k++) work();
```

Q12. Identify the time complexity.

```
for (int i = 1; i < n; i *= 3) work();
```

Q13. What is the auxiliary space complexity?

```
int total = 0;
for (int i = 0; i < n; i++) total += i;
```

Q14. What is the space complexity of storing n integers in an additional array?

```
int *values = allocate_array_of_size(n);
```

Q15. What are the time and auxiliary space complexities?

```
void countDown(int n) {
    if (n == 0) return;
    countDown(n - 1);
}
```

Q16. A program performs 7n + 50 basic operations. Give its Big-O complexity.

Answer: _____

Q17. A program has two sequential blocks with complexities $O(n^2)$ and $O(n^3)$. What is the overall complexity?

Answer: _____

Q18. Explain why two nested loops do not automatically imply $O(n^2)$. Give an example.

Answer: _____

Q19. Arrange in increasing order of growth: $O(n^2)$, $O(1)$, $O(n \log n)$, $O(\log n)$, $O(2^n)$, $O(n)$.

Answer: _____

Q20. An algorithm stores an n × n matrix and then scans every cell once. Give its time and auxiliary space complexities, excluding other memory.

Answer: _____

10. Answer key

Try the questions before checking this section.

- Q1. Time $O(1)$.
- Q2. Time $O(n)$.
- Q3. Time $O(n^2)$.
- Q4. Time $O(\log n)$.
- Q5. Time $O(n \log n)$.
- Q6. $O(n + n) = O(2n) = O(n)$.
- Q7. $O(10n) = O(n)$.
- Q8. $O(n^2)$.
- Q9. $O(n)$, because n dominates $\log n$.
- Q10. $O(\sqrt{n})$.
- Q11. $O(n^2 + n) = O(n^2)$.
- Q12. $O(\log n)$.
- Q13. $O(1)$ auxiliary space.
- Q14. $O(n)$ space.
- Q15. Time $O(n)$; auxiliary stack space $O(n)$.
- Q16. $O(n)$.
- Q17. $O(n^3)$, the dominant term.
- Q18. The inner loop may run a fixed number of times; for example, n outer iterations × 10 inner iterations = $10n = O(n)$.
- Q19. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- Q20. Time $O(n^2)$; auxiliary space $O(n^2)$ if the matrix is additional memory allocated by the algorithm. If the matrix is the input, auxiliary space for scanning can be $O(1)$.

Key takeaway: Do not guess complexity by counting loops. Count how the work and memory grow with input size.